# Rocky Linux 8.10におけるOpenFXプラグイン開発：エンジニアリング・詳細ガイド

ビジュアル・エフェクト（VFX）およびポストプロダクション業界において、ソフトウェアの相互運用性は長年の課題であった。この課題を解決するために提唱された標準規格がOpenFX（別名OFX）である。OpenFXは、2Dビジュアル・エフェクトやコンポジッティング・プラグインのためのオープン標準であり、この規格に準拠して書かれたプラグインは、規格をサポートするあらゆるホストアプリケーション上で動作することが可能となる[1]。本報告書では、エンタープライズLinuxの堅牢な基盤を持つRocky Linux 8.10を対象とした、OpenFXプラグイン開発の詳細な技術ガイドを提供する。

Rocky Linux 8.10は、Red Hat Enterprise Linux (RHEL) 8系列の最終マイナーリリースであり、今後5年間にわたるセキュリティアップデートが保証されているため、プロフェッショナルな制作環境における開発プラットフォームとして極めて高い信頼性を有している[2]。本ガイドでは、環境構築からAPIアーキテクチャ、レンダリング処理、マルチスレッド最適化、そしてパッケージングに至るまで、専門的な知見に基づいた詳細な解説を行う。

## 開発環境の構築とシステム基盤

Rocky Linux 8.10での開発において、まず考慮すべきはコンパイラ・ツールチェーンとライブラリの整合性である。VFX業界では「VFX Reference Platform」と呼ばれる標準規格が毎年更新されており、開発環境はこのガイドラインに準拠することが望ましい[4]。

### コンパイラおよびビルドツールの選定

Rocky Linux 8.10のデフォルトのGCCバージョンは8.5.0であるが、現代的なOpenFX開発においてはC++17以降の言語機能が要求されることが多い[6]。これに対応するため、AppStreamリポジトリ経由で提供される「GCC Toolset」を利用することが標準的である。GCC Toolsetは、システムの基本ライブラリを破壊することなく、最新のGCC、GDB、およびパフォーマンス解析ツールを提供するための仕組みである[8]。

| コンポーネント | バージョン / 指定方法 | 備考 |
|---|---|---|
| OS | Rocky Linux 8.10 | 100% RHELバイナリ互換[2] |
| デフォルトGCC | 8.5.0 | 基本的なビルドに使用[6] |
| 推奨GCC Toolset | GCC Toolset 13 | C++20/23対応、最新の最適化[10] |

| | | |
|---|---|---|
| CMake | 3.28以上を推奨 | ビルド構成の自動化 [11] |
| Conan | 1.x (2.x移行期) | パッケージ管理ツール [11] |
| glibc | 2.28 | システムの基幹Cライブラリ [5] |

GCC Toolsetを有効化するには、scl（Software Collections）コマンドを使用する。例えば、GCC Toolset 13を使用してシェルセッションを開始する場合は、scl enable gcc-toolset-13 bashを実行する。これにより、現在のセッションにおいてパスが優先的に設定される [9]。

### 必須依存ライブラリのインストール

OpenFX自体のSDKはC言語のヘッダーファイル群で構成されているが、実際のプラグイン開発にはグラフィックス関連のライブラリやビルドユーティリティが必要となる。以下の表に、Rocky Linux 8.10でインストールすべき主要なパッケージを示す。

| パッケージ名 | **DNF**インストールコマンド | 用途 |
|---|---|---|
| Development Tools | dnf groupinstall "Development Tools" | make, gcc, autoconf等の基本ツール [9] |
| mesa-libGL-devel | dnf install mesa-libGL-devel | OpenGLレンダリングおよびインターフェース [15] |
| libX11-devel | dnf install libX11-devel | X Window Systemとの連携 [16] |
| Python 3.12 | dnf install python3.12 | ビルドスクリプトおよび自動化 [3] |
| expat-devel | dnf install expat-devel | XMLリソースの解析 [17] |

また、VFX制作現場では共有ストレージやレンダーファームとの連携が不可欠であるため、NFS（Network File System）環境での権限設定にも注意が必要である。特に/optディレクトリなどにツールセットを配置する場合、ルート権限の制限（root_squash）がビルド処理に影響を与える可能性がある [18]。

## OpenFXアーキテクチャの核心

OpenFXの設計思想は、ホスト（Host）とプラグイン（Plug-in）の間の「疎結合」にある。ホストはプラグインに対して機能のセットである「スイート（Suites）」を公開し、プラグインはホストからの「アクション（

Actions）」に応答することで動作する[1]。

## スイート（Suites）とアクション（Actions）のメカニズム

APIの実体はC言語の構造体であり、ホストは関数ポインタのテーブルをプラグインに渡す。これにより、C++のシンボルマングリング問題やABIの不整合を回避し、異なるコンパイラでビルドされたアプリケーション間での動的リンクを可能にしている[19]。

アクションはC文字列（const char *）として定義されており、プラグインのメインエントリポイント関数に渡される。この設計により、将来的なAPIの拡張において新しいアクションが追加されたとしても、古いプラグインとの後方互換性を維持することが容易となっている[19]。

| アクション名 | 実行タイミング / 役割 | 渡されるハンドル |
|---|---|---|
| kOfxActionLoad | バイナリ読み込み直後 | NULL[21] |
| kOfxActionDescribe | プラグインの能力定義 | 記述子ハンドル[19] |
| kOfxActionCreateInstance | エフェクトの適用時 | インスタンスハンドル[21] |
| kOfxImageEffectActionRender | フレームの計算時 | インスタンスハンドル[23] |
| kOfxActionUnload | バイナリの解放前 | NULL[21] |

## エントリポイントの定義

OpenFXプラグインは、共有ライブラリ（DSO）としてビルドされ、ホストに対して以下の2つのシンボルを公開しなければならない。

1. OfxGetNumberOfPlugins: ライブラリに含まれるプラグインの数を返す。
2. OfxGetPlugin: インデックスに対応するプラグインの構造体（OfxPlugin）を返す。

プラグイン構造体には、APIの名称、バージョン、およびメインエントリポイント関数のポインタが含まれる[19]。メインエントリポイントは以下のシグネチャを持つ。

C

```
OfxStatus OfxPluginEntryPoint(const char *action, const void *handle, OfxPropertySetHandle inArgs,
```

```
OfxPropertySetHandle outArgs);
```

この関数内で、渡されたaction文字列に応じた条件分岐を行い、必要な処理を実行することがプラグイン開発の基本構造となる[19]。

# プラグインのライフサイクルと初期化処理

OpenFXプラグインは厳格なライフサイクルに従って動作する。このプロセスの各段階を正しく実装することは、メモリリークの防止や安定性の確保において極めて重要である。

## ロードと記述フェーズ

バイナリがメモリにロードされると、最初にkOfxActionLoadが呼び出される。ここでは、グローバルなデータ構造の初期化や、ホストから提供される基本スイート（Property Suite, Parameter Suiteなど）の取得が行われる[21]。

続いて実行されるkOfxActionDescribeは、プラグインがその「正体」をホストに宣言する段階である。このアクションでは、プラグインがサポートするピクセル深度（8ビット整数、16ビット整数、32ビット浮動小数点など）、コンポーネント（RGBA, Alphaのみ等）、およびマルチスレッド性能について記述する[19]。ここでの記述は「記述子（Descriptor）」に対して行われ、実際の個別のエフェクト設定とは独立している[19]。

## コンテキストに応じた記述

OpenFXには「コンテキスト（Context）」という概念があり、プラグインがどのような環境で使用されるかを定義する。例えば、「Filter」コンテキストは1つの入力と1つの出力を持ち、「General」コンテキストは任意の数の入力をサポートする[26]。

| コンテキスト名 | 特徴 | 典型的な用途 |
|---|---|---|
| kOfxImageEffectContextFilter | 入力1、出力1 | 色補正、ブラー、ノイズ付加[26] |
| kOfxImageEffectContextGeneral | 任意の入力数、カスタム名称 | 合成、複雑なマスキング処理[26] |
| kOfxImageEffectContextTransition | 入力2（Source/Target） | クロスフェード、ワイプ[22] |

kOfxImageEffectActionDescribeInContextアクションでは、特定のコンテキストにおいて必要となるクリップ（Clips）やパラメータの詳細を定義する。これにより、一つのバイナリで簡易的なフィルタとし

ても、高度なコンポジットツールとしても動作する柔軟なプラグインを実現できる[25]。

# 画像処理の基礎とメモリ管理

プラグインの主目的は画像データの変換である。OpenFXにおける画像処理は、ピクセルデータへのダイレクトアクセスと、座標系の適切な理解が不可欠である。

## クリップ、イメージ、ピクセルデータ

「クリップ（Clip）」は画像のストリームを表し、実際の画像データは「イメージ（Image）」として取得される。レンダリング中、プラグインはclipGetImage関数を介して特定の時間のイメージをフェッチする[23]。取得されたイメージはプロパティセットであり、ピクセルデータへのポインタ（kOfxImagePropData）や、各走査線のバイト幅（kOfxImagePropRowBytes）などの属性を保持している[22]。

OpenFXのピクセルデータは通常「チャンキー（Chunky）」形式で配置される。例えばRGBA形式の場合、メモリ上の配置は以下のようになる。

$$P_{x,y} = \{R_{x,y}, G_{x,y}, B_{x,y}, A_{x,y}\}$$

ピクセルのアドレス計算は以下の公式に従う。

$$\text{Address}(x, y) = \text{BasePointer} + (y \times \text{RowBytes}) + (x \times \text{PixelBytes})$$

ここで、PixelBytesはチャンネル数とピクセル深度によって決まる定数である。Rocky Linux 8.10のような64ビットシステムでは、メモリアライメントを考慮した高速なメモリアクセスがパフォーマンスに直結する[22]。

## 座標系と空間的認識

OpenFXは、解像度に依存しない「カノニカル座標系（Canonical Coordinate System）」を採用している[23]。これにより、異なる解像度の素材を組み合わせる際の計算が簡略化される。また、「ピクセル・アスペクト比（Pixel Aspect Ratio）」を考慮することで、アナモルフィック・レンズで撮影された素材などの歪みを正しく処理できる[22]。

| 座標属性 | 意味 | 関連プロパティ |
|---|---|---|
| ROD (Region of Definition) | 画像が存在する全領域 | kOfxImagePropRegionOfDefinition[23] |
| ROI (Region of Interest) | 処理に必要な入力領域 | kOfxImageEffectActionGetRegionsOfInterest[23] |

| | | |
|---|---|---|
| Render Window | 現在レンダリング中の具体的な窓領域 | kOfxImageEffectPropRenderWindow [24] |

レンダリングアクション（kOfxImageEffectActionRender）に際して、ホストはプラグインに対し、特定の領域（Render Window）の描画を要求する。プラグインはこの範囲外を描画する必要はなく、逆に範囲外を要求された場合は黒（透明）で塗りつぶすことが求められる [23]。

# マルチスレッドレンダリングと最適化

現代のVFXプロダクションにおいて、CPUの全コアを活用した並列処理は必須要件である。OpenFXは、ホスト主導とプラグイン主導の両面でマルチスレッドをサポートしている。

## スレッドセーフティの宣言

プラグインは記述段階で、自身のスレッド安全性のレベルをホストに通知する。これにより、ホストはどのようにレンダリングアクションをスケジュールするかを決定する。

1. **kOfxImageEffectRenderUnsafe**: 同時に一つのインスタンスしかレンダリングできない。古いライブラリを使用する場合などに指定する [23]。
2. **kOfxImageEffectRenderInstanceSafe**: 異なるインスタンスであれば並列実行可能。パラメータ変更とレンダリングが別スレッドで行われる可能性がある [23]。
3. **kOfxImageEffectRenderFullySafe**: 同一インスタンス内でも、異なるRender Windowであれば完全に並列実行可能 [23]。

## OfxMultiThreadSuiteの活用

プラグイン内部で並列化を行う場合、ホストが提供するOfxMultiThreadSuiteを利用することが推奨される [29]。このスイートは、OSレベルのスレッド生成を抽象化し、ホストが管理するスレッドプールを利用してSMP（対称型マルチプロセッシング）実行を可能にする。

典型的な実装パターンでは、画像のY方向の走査線をスレッド数に応じて分割し、各スレッドに割り当てる。

C++

```
void multiThreadFunction(unsigned int threadId, unsigned int nThreads, void *customArg) {
  // 全体のY範囲をnThreadsで分割し、threadIdに対応する範囲を処理する [29, 31]
}
```

この手法により、Rocky Linux 8.10が動作するマルチコア・ワークステーションの性能を最大限に引き出すことができる。また、クリティカルセクションを保護するために、スイート内のmutexCreate、

mutexLock等を利用した排他制御も可能である[29]。

# パラメータ・システムとプロパティの管理

プラグインの柔軟性は、ユーザーが調整可能な「パラメータ」によって決まる。OpenFXのパラメータシステムは、強力なプロパティ・メカニズムに基づいている。

## パラメータの定義とUI構成

パラメータは整数、浮動小数点、文字列、ブール値、プッシュボタン、カスタムタイプなど多岐にわたる。これらはkOfxActionDescribeInContext内で定義され、階層構造（Group）やページ（Page）によって整理される[32]。

| パラメータ属性 | 役割 | 設定例 |
|---|---|---|
| Label | UIに表示される名称 | "Contrast", "Saturation"[32] |
| Hint | ユーザー向けの説明文 | "Adjusts the brightness difference"[19] |
| Default | 初期値 | 1.0, 0, false[19] |
| Range | 有効な値の範囲 | 0.0 to 10.0[19] |

## インタラクティブな応答

ユーザーがUI上でパラメータを変更すると、ホストはkOfxActionInstanceChangedアクションを送信する。プラグインはこの通知を受け取り、他のパラメータの表示/非表示を切り替えたり、値を動的に再計算したりすることができる[28]。例えば、あるチェックボックスがオンになった時だけ詳細設定を表示するといった、動的なユーザーインターフェースの構築が可能である[22]。

# 高度な機能：GPUレンダリングとインターアクト

OpenFXはCPUレンダリングにとどまらず、ハードウェア加速や高度なユーザー・インタラクションをサポートするための拡張を備えている。

## CUDAおよびOpenCLによるGPU加速

高負荷な画像処理において、GPUの活用は劇的な速度向上をもたらす。OpenFX 1.5以降、CUDA、OpenCL、MetalなどのGPU APIとの親和性が高められている[1]。

- **CUDA**: ホストはプラグインに対し、cudaStream_tポインタを渡す。プラグインは自身のカーネルをこのストリーム上で実行し、非同期に処理を完結させることが期待される[24]。

- **OpenCL**: 同様に、OpenCLコマンドキューが共有される。Rocky Linux 8.10環境では、NVIDIAの公式ドライバやocl-icd等のライブラリを適切に設定することで、これらの機能を利用可能になる[11]。

### インターアクト（OpenGLオーバーレイ）

プラグインがホストのビューワー上に独自の操作ハンドルや描画を行いたい場合、「インターアクト（Interacts）」機能を使用する[1]。これはOpenGLを利用してフレーム上にグラフィックスをオーバーレイする仕組みである。

kOfxInteractActionDrawアクションが発生すると、ホストは適切なOpenGLコンテキストをカレントにし、プラグインに描画を許可する[24]。また、マウスの移動（kOfxInteractActionPenMotion）やクリック（kOfxInteractActionPenDown）などのイベントを処理することで、画面上のハンドルをドラッグしてパラメータを操作するような直感的なUIを提供できる[1]。

## Rocky Linuxにおけるパッケージングとデプロイメント

開発したプラグインを配布・利用するためには、OpenFXの標準的なディレクトリ構造に従ってパッケージングする必要がある。

###.ofx.bundleの構造

LinuxにおけるOpenFXプラグインは、単一の共有ライブラリファイルではなく、特定のディレクトリ構造を持つ「バンドル（Bundle）」として提供される[35]。

/usr/OFX/Plugins/MyEffect.ofx.bundle/

└── Contents/

├── Linux-x86_64/

│ └── MyEffect.ofx (実際の共有ライブラリ)

└── Resources/

├── MyEffect.png (アイコン)

└── MyEffect.xml (追加のリソース)

この構造により、一つのバンドル内に複数のアーキテクチャ（x86_64, aarch64等）のバイナリを含めることが可能となる[35]。バイナリの拡張子は、Linux上であっても.ofxとするのが規格上の規定である[35]。

### シンボル可視性とリンクの最適化

LinuxでのDSO開発において、不必要なシンボルの露出はロード時間の増大や名称衝突を招く。OpenFXプラグインでは、エントリポイントとなる2つの関数以外はすべて「非表示（Hidden）」にすることが推奨される [37]。

GCCのコンパイルオプションとして-fvisibility=hiddenを指定し、公開すべき関数にのみ__attribute__((visibility("default")))を付与する。これにより、ELFシンボルテーブルが整理され、Boost.Pythonなどの巨大なライブラリをリンクしている場合でも、ロード時間を大幅に短縮できる [37]。

### VFX Reference Platformへの準拠

プロフェッショナルなVFX環境において、プラグインが他のライブラリ（Python, Qt, Boost等）と競合しないことは極めて重要である。VFX Reference Platformは、特定の年度におけるこれらの標準バージョンを規定している [4]。

| 年度ターゲット | C++標準 | glibc | Python |
|---|---|---|---|
| CY2024 | C++17 | 2.28 | 3.11 [7] |

Rocky Linux 8.10の基本ライブラリであるglibc 2.28は、CY2024の要件を満たしており、GCC Toolset 13を利用することでC++17/20のコンパイル要件もクリアできる [3]。

## デバッグと診断手法

複雑なホストアプリケーション内で動作する共有ライブラリのデバッグは、スタンドアロンアプリケーションとは異なるアプローチが必要となる。

### GDBによるホストプロセスへのアタッチ

Rocky Linux 8.10で最も一般的なデバッグ手法は、GDB（GNU Debugger）を使用して、プラグインをロードしているホストプロセス（Nuke, Natron, DaVinci Resolve等）にアタッチすることである [6]。

1. ホストアプリケーションを通常通り起動する。
2. ps -aux | grep appnameでプロセスIDを確認する。
3. gdb -p PIDでアタッチし、ソースコードの適切な場所にブレークポイントを設定する。

あるいは、IDE（VSCodeやCLion）のデバッグ設定で「外部実行ファイル」としてホストを指定し、プラグインプロジェクトから直接起動させる方法も効率的である [40]。

### ofxTestLog.txtと診断出力

OpenFX C++サポートライブラリには、デバッグビルド時に詳細なログを出力する機能が組み込まれている。これは通常、カレントディレクトリにofxTestLog.txtとして生成される [41]。

このログには、ホストが提供するプロパティの妥当性確認（Property Validation）の結果が記録され

る。もしホストが規格外のプロパティ値を返していたり、必須のプロパティが欠落していたりする場合、このログを確認することで原因を迅速に特定できる[43]。また、標準出力へのprintfやstd::coutは、ホストをターミナルから起動していれば直接確認することが可能である[41]。

# 結論とエンジニアリング上の提言

OpenFXは、Rocky Linux 8.10という安定したエンタープライズ環境において、その真価を発揮する強力な標準規格である。規格自体はC言語ベースのシンプルなインターフェースを採用しているが、その背後にあるマルチスレッド管理、座標系変換、GPU加速といった要素は高度な専門知識を要求する。

開発にあたっては、まずGCC Toolsetによる現代的な開発環境を構築し、VFX Reference Platformのガイドラインを遵守することが、安定した商用プラグインへの第一歩となる。また、単なる画像変換処理にとどまらず、インターアクト機能を通じた直感的な操作性の提供や、OpenMfxのような3Dメッシュ処理への将来的な拡張を視野に入れることで、より付加価値の高いツール開発が可能となる[1]。

本ガイドが、Rocky Linux 8.10におけるOpenFXプラグイン開発の強固な基盤となり、ビジュアル・エフェクト業界のさらなる技術革新に寄与することを期待する。

引用文献


1. OpenFX (API) - Wikipedia, 2月 23, 2026にアクセス、https://en.wikipedia.org/wiki/OpenFX_(API)
2. Rocky Linux 8.10 Available Now, 2月 23, 2026にアクセス、https://rockylinux.org/news/rocky-linux-8-10-ga-release
3. Release notes for Rocky Linux 8.10, 2月 23, 2026にアクセス、https://docs.rockylinux.org/10/release_notes/8_10/
4. Linux in VFX - VFX Reference Platform, 2月 23, 2026にアクセス、https://vfxplatform.com/linux/
5. Frequently Asked Questions - VFX Reference Platform, 2月 23, 2026にアクセス、https://vfxplatform.com/FAQ/
6. "Development Tools" DNF group installed but group packages arent? - Rocky Linux Forum, 2月 23, 2026にアクセス、https://forums.rockylinux.org/t/development-tools-dnf-group-installed-but-group-packages-arent/5913
7. Platform History | VFX Reference Platform, 2月 23, 2026にアクセス、https://vfxplatform.com/platform_history.html
8. Chapter 4. Additional toolsets for development | Developing C and C++ applications in RHEL 8 | Red Hat Enterprise Linux, 2月 23, 2026にアクセス、https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/developing_c_and_cpp_applications_in_rhel_8/additional-toolsets-for-development_developing-applications
9. Install GCC and build a Hello World application on RHEL 8 - Red Hat Developer, 2月 23, 2026にアクセス、https://developers.redhat.com/articles/2023/11/10/install-gcc-and-build-hello-wor

ld-application-rhel-8
10. G++ and libsupc++ - Rocky Linux Help & Support, 2月 23, 2026にアクセス、https://forums.rockylinux.org/t/g-and-libsupc/14243
11. openfx/install.md at main - GitHub, 2月 23, 2026にアクセス、https://github.com/AcademySoftwareFoundation/openfx/blob/main/install.md
12. Preparing Open RV on Rocky 8 and 9, 2月 23, 2026にアクセス、https://aswf-openrv.readthedocs.io/en/latest/build_system/config_linux_rocky89.html
13. lswenzz/OpenFX: OpenFX image processing plug-in standard. - GitHub, 2月 23, 2026にアクセス、https://github.com/lswenzz/OpenFX
14. VFX Reference Platform compatibility - AWS Deadline Cloud, 2月 23, 2026にアクセス、https://docs.aws.amazon.com/deadline-cloud/latest/userguide/smf-vfx.html
15. Mesa-libGL-devel Download for Linux (rpm), 2月 23, 2026にアクセス、https://pkgs.org/download/mesa-libGL-devel
16. LibX11-devel(x86-64) Download for Linux (rpm), 2月 23, 2026にアクセス、https://pkgs.org/download/libX11-devel(x86-64)
17. CMakeLists.txt - AcademySoftwareFoundation/openfx - GitHub, 2月 23, 2026にアクセス、https://github.com/AcademySoftwareFoundation/openfx/blob/main/CMakeLists.txt
18. Dnf upgrade fails (gcc-toolset-13, cpio: symlink failed) - Rocky Linux Help & Support, 2月 23, 2026にアクセス、https://forums.rockylinux.org/t/dnf-upgrade-fails-gcc-toolset-13-cpio-symlink-failed/17640
19. Structure of The OFX and the Image Effect API - OpenFX documentation, 2月 23, 2026にアクセス、https://openfx.readthedocs.io/en/main/Reference/ofxStructure.html
20. OpenFX - Read the Docs, 2月 23, 2026にアクセス、https://buildmedia.readthedocs.org/media/pdf/openfx/latest/openfx.pdf
21. Actions Passed to a Mesh Effect - OpenMfx documentation, 2月 23, 2026にアクセス、https://openmesheffect.org/Reference/ofxMeshEffectActions.html
22. OpenFX Programming Guide, 2月 23, 2026にアクセス、https://openfx.readthedocs.io/en/main/Guide
23. openfx/Documentation/sources/Reference/ofxRendering.rst at main - GitHub, 2月 23, 2026にアクセス、https://github.com/ofxa/openfx/blob/master/Documentation/sources/Reference/ofxRendering.rst
24. Rendering — OpenFX 1.5.1 documentation, 2月 23, 2026にアクセス、https://openfx.readthedocs.io/en/main/Reference/ofxRendering.html
25. OpenFX documentation, 2月 23, 2026にアクセス、https://openfx.readthedocs.io/_/downloads/en/main/epub/
26. Multiple Contexts, Why Bother? — OpenFX 1.5.1 documentation, 2月 23, 2026にアクセス、https://openfx.readthedocs.io/en/main/Guide/ofxExample4_Saturation.html
27. Pixel Processing - Join Phase | A Trip through the Graphics Pipeline, 2月 23, 2026

にアクセス、
https://alaingalvan.gitbook.io/a-trip-through-the-graphics-pipeline/chapter9-pixel-processing-join
28. Thread and Recursion Safety - OpenFX documentation - Read the Docs, 2月 23, 2026にアクセス、
https://openfx.readthedocs.io/en/main/Reference/ofxThreadSafety.html
29. OfxMultiThreadSuiteV1 — OpenFX 1.5.1 documentation, 2月 23, 2026にアクセス、
https://openfx.readthedocs.io/en/main/Reference/suites/ofxThreadingSuiteReference.html
30. openfx/include/ofxMultiThread.h at main - GitHub, 2月 23, 2026にアクセス、
https://github.com/AcademySoftwareFoundation/openfx/blob/master/include/ofxMultiThread.h
31. openfx/Examples/Basic/basic.cpp at main · AcademySoftwareFoundation/openfx - GitHub, 2月 23, 2026にアクセス、
https://github.com/AcademySoftwareFoundation/openfx/blob/master/Examples/Basic/basic.cpp
32. Releases · AcademySoftwareFoundation/openfx · GitHub, 2月 23, 2026にアクセス、
https://github.com/AcademySoftwareFoundation/openfx/releases
33. janimatic/ofxSample: Example of cmake based openfx project : easy openfx plugin dev for beginners - GitHub, 2月 23, 2026にアクセス、
https://github.com/janimatic/ofxSample
34. Packaging OFX Plug-ins — OpenFX 1.5.1 documentation, 2月 23, 2026にアクセス、
https://openfx.readthedocs.io/en/main/Reference/ofxPackaging.html
35. Official FAQ Friday: Plugins : r/davinciresolve - Reddit, 2月 23, 2026にアクセス、
https://www.reddit.com/r/davinciresolve/comments/ksvl91/official_faq_friday_plugins/
36. Building with -fvisibility=hidden fails on linking · Issue #665 · PixarAnimationStudios/OpenUSD - GitHub, 2月 23, 2026にアクセス、
https://github.com/PixarAnimationStudios/USD/issues/665
37. Visibility - GCC Wiki, 2月 23, 2026にアクセス、https://gcc.gnu.org/wiki/Visibility
38. Gcc hide visibility of symbols coming from included static library - Stack Overflow, 2月 23, 2026にアクセス、
https://stackoverflow.com/questions/31670950/gcc-hide-visibility-of-symbols-coming-from-included-static-library
39. Debugging OpenFX plugins - Stack Overflow, 2月 23, 2026にアクセス、
https://stackoverflow.com/questions/36889897/debugging-openfx-plugins
40. OpenFX Plugin Development/Debugging on Windows - Natron - discuss.pixls.us, 2月 23, 2026にアクセス、
https://discuss.pixls.us/t/openfx-plugin-development-debugging-on-windows/7720
41. CMakeLists.txt | CLion Documentation - JetBrains, 2月 23, 2026にアクセス、
https://www.jetbrains.com/help/clion/cmakelists-txt-file.html
42. openfx/Support/README at main - GitHub, 2月 23, 2026にアクセス、
https://github.com/AcademySoftwareFoundation/openfx/blob/master/Support/README